The 3-Layer Industrial World Model Stack: Why AI Labs Are Building One-Third of What Industrial Autonomy Requires

Pieter van Schalkwyk[1]
CEO, XMPro Inc.
ORCID: 0009-0009-3982-9346
pvs@xmpro.com

Gavin Green[1]
VP Strategic Solutions, XMPro Inc.
ORCID: 0009-0007-8315-5559
gavin.green@xmpro.com

[1]XMPro Inc., Dallas, Texas, USA

March 2026

Abstract

AMI Labs (founded by Yann LeCun) raised approximately $1.03 billion in 2026 to build world-model-based AI systems. Meta's V-JEPA 2 programme demonstrated that self-supervised video models can learn physical reasoning transferable to robotic manipulation with on the order of tens of hours of robot-specific data. World Labs (Fei-Fei Li) is building Marble, a model that generates explorable 3D worlds and persistent scene representations that respond to actions. These programmes share a common focus: world models (AI systems that predict how environments evolve under actions) as the foundation for autonomous systems.

This paper argues that these programmes address only one of three layers that industrial autonomy requires. A refinery, mine, or pharmaceutical plant cannot be safely operated by an AI that understands physics alone. It must also understand operational key performance indicators, digital twin dynamics, maintenance risk, alarm behaviour, safety procedures, business constraints, and human organisational patterns.

We propose that an industrial world model must be a **three-layer stack**: (1) a *Physical Dynamics Layer* that predicts how physical systems and materials evolve under control actions; (2) an *Operational KPI Dynamics Layer* that translates physical state evolution into production value metrics; and (3) a *Socio-Technical Dynamics Layer* that evaluates whether a physically and economically optimal strategy is acceptable given human, safety, regulatory, and business constraints.

The paper makes six contributions: (1) a formal definition of the three-layer prediction stack with clean API boundaries; (2) an extended Layer 3 specification requiring explainable risk assessment and adaptive constraint learning; (3) an analysis of the knowledge acquisition problem — how Layer 2 and 3 knowledge enters the system and remains current; (4) a specification of the cognitive agent requirement — the observe–reflect–plan–act reasoning layer that operates over the prediction stack and is necessary for translating prediction into autonomous action; (5) concrete end-to-end demonstrations on both a copper grinding-flotation circuit and an upstream oil & gas separation train; and (6) a progressive autonomy framework with formal technical gate criteria for each phase transition. The architecture aligns with LeCun's AMI blueprint and defines a vendor-agnostic interface for plugging in foundation world models as they become available for industrial deployment.

Keywords: world models, industrial autonomy, multi-agent systems, cognitive agents, digital twins, advanced process control, agentic operations, JEPA, mineral processing

1 Introduction: The World Model Moment

Yann LeCun has argued for some years that large language models cannot by scaling alone acquire grounded physical cause-and-effect understanding (7). His position is that world models (systems that predict in abstract representation space rather than token or pixel space) are the necessary foundation for machine intelligence that can act reliably in physical environments. AMI Labs raised approximately $1.03 billion in 2026 to pursue this direction (1). Meta's V-JEPA 2 programme demonstrated that self-supervised video prediction models can learn physical reasoning transferable to robotic manipulation with on the order of tens of hours of robot-specific data (around 62 hours in reported experiments) (9). World Labs' Marble model generates explorable 3D worlds and persistent scene representations that respond to actions (14).

For industrial operations, this matters. Autonomous plant operation (a system that perceives a complex industrial environment, reasons about cause and effect, plans safe and valuable actions, and acts within defined authority) has been out of reach because the physical reasoning foundation was not there. These programmes are building that foundation.

The problem is that physical reasoning is Layer 1 of three. The programmes described above are building the physical dynamics layer. For autonomous industrial operation, two additional layers are required. This paper defines all three, demonstrates the complete stack on a copper grinding-flotation circuit, and proposes a practical implementation path using existing industrial assets.

2 The Industrial Specificity Problem

2.1 What Physical World Models Can and Cannot Do

The world model programmes at AMI Labs, Meta, and World Labs share a common design focus: they learn how physical environments evolve when actions are applied. They capture how objects move and collide, how materials flow and deform, how spatial configurations change under manipulation. This is substantive progress on a hard problem.

What these models do not address is where the industrial specificity problem begins. In a refinery, mine, or pharmaceutical plant, autonomous control requires prediction at three levels:

1. **Physical prediction:** If I increase mill speed by 2% and raise water addition, what happens to particle size distribution, mill power draw, and cyclone feed density over the next 30 minutes?
2. **Operational prediction:** Given those physical changes, what happens to plant throughput, concentrate grade, copper recovery, energy per tonne, and risk of process instability over the next two hours?
3. **Socio-technical prediction:** Given those KPI changes, will this strategy trigger an alarm flood during the ore hardness transition at hour two? Will it accelerate liner wear in a way that wipes out a week of throughput gains? Does it risk violating a contractual shipment specification?

A system that answers question 1 but not questions 2 or 3 cannot be deployed as an autonomous agent on a production circuit. The gap between physical validity and operational acceptability can represent millions of dollars in production loss, safety incidents, or contractual penalties.

2.2 Why the Gap Exists

The three-layer stack remains incomplete because of a structural misalignment between the communities building world models and those operating industrial plants. Table 1 summarises the situation.

Table 1: Structural gap between world model research and industrial autonomy

Community	**Current Focus**	**Gap**
AI Research Labs (AMI, World Labs, Meta)	Self-supervised video, 3D scene, and RL benchmarks. Physical reasoning and embodied AI.	Operational and socio-technical dynamics out of scope. Industry integration assumed to follow.
Industrial Software Vendors	Digital twins and APC for specific control loops. Dashboard-centric.	No unified API for autonomous agents. Layers mixed. Human dynamics not modelled.
Industrial Enterprises	Operational efficiency and safety. Specific digital twin and APC projects.	No architectural framework to compose physical, operational, and organisational models for agent use.

The opportunity is to define the full three-layer stack explicitly, enabling each community to focus on what it does best while the composed architecture delivers capabilities none could achieve alone.

3 The 3-Layer Industrial World Model Stack

3.1 Architecture Overview

We define an industrial world model as a three-layer composition of prediction services, each answering a distinct class of question. Cognitive agents simulate candidate actions through all three layers before acting. The information flow is:

$$\text{Perception} \rightarrow \underbrace{\mathcal{L}_1 \rightarrow \mathcal{L}_2 \rightarrow \mathcal{L}_3}_{\text{Prediction Stack}} \rightarrow \underbrace{\mathcal{A}}_{\text{Cognitive Agent}} \rightarrow \text{Recommendation} \tag{1}$$

where $\mathcal{L}_1$, $\mathcal{L}_2$, and $\mathcal{L}_3$ are the Physical Dynamics, Operational KPI Dynamics, and Socio-Technical Dynamics layers respectively, and $\mathcal{A}$ is the cognitive agent layer that generates candidate strategies, simulates them through the prediction stack, reasons over the results, and produces ranked recommendations with explanations. The prediction stack provides calibrated foresight; the cognitive agent provides the reasoning and decision-making that translates foresight into action. Table 2 provides a high-level comparison of the prediction layers.

Table 2: The 3-Layer Industrial World Model Stack

Layer	Name	Core Prediction Question	Who Is Building It
3	Socio-Technical / Organisational	Even if KPIs improve, will this strategy cause unacceptable human, safety, or business side-effects?	Not addressed in ML literature
2	Operational / KPI Dynamics	Given process conditions and setpoints, what happens to throughput, grade, recovery, energy?	Industry needs but no systematic build
1	Physical / World Dynamics	Given current physical state and control actions, what happens next physically?	AMI Labs, World Labs, Meta V-JEPA 2

Core thesis: An industrial world model must be a three-layer stack. AI labs are building Layer 1 (physical dynamics). Nobody is systematically building Layers 2 and 3 (operational KPI dynamics and socio-technical constraints). Without all three layers, world-model-based agents cannot be safely deployed on production industrial systems.

Figure 1 illustrates the architecture. Agents simulate candidate actions through all three layers sequentially before any action is taken or recommended.

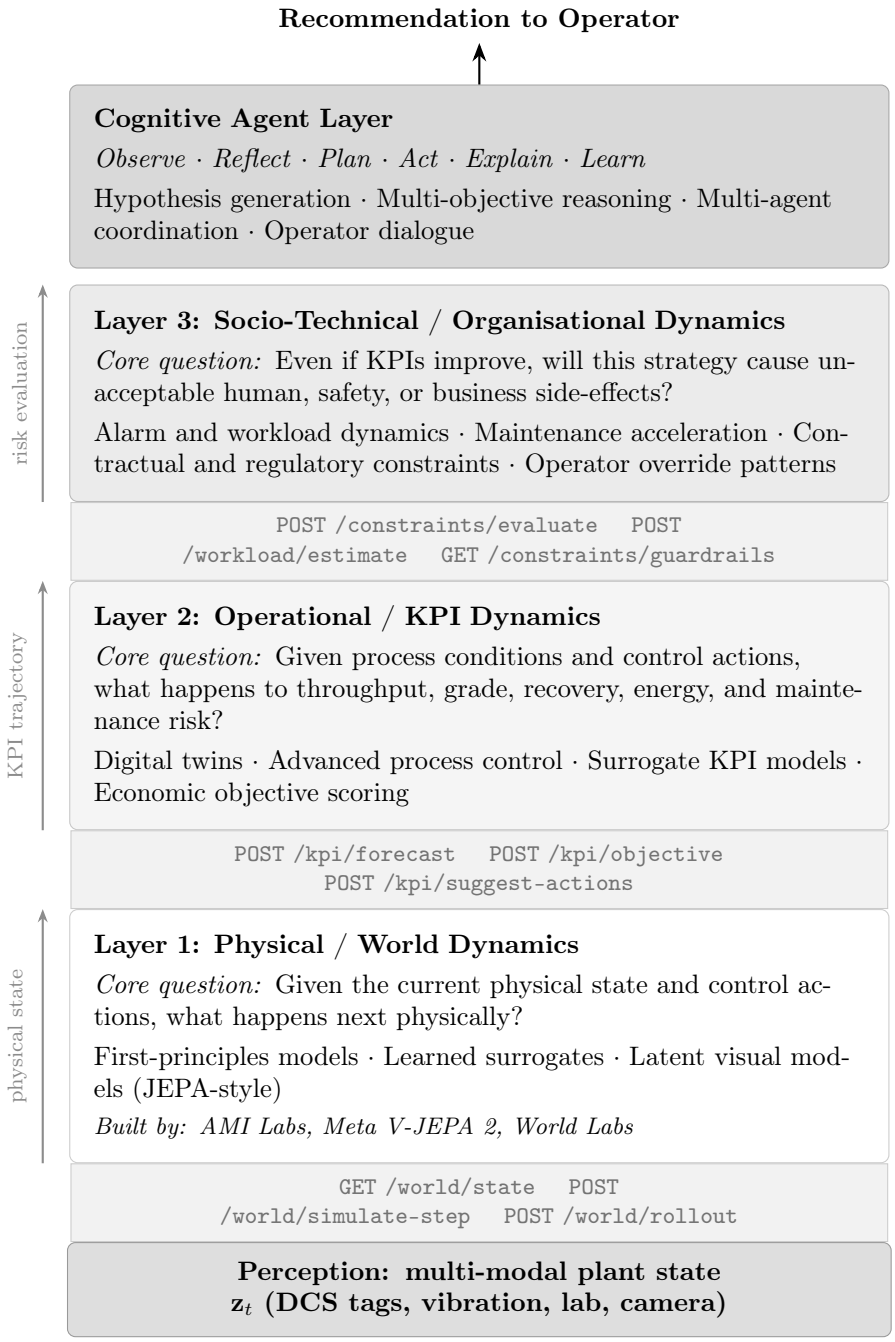

Figure 1: The 3-Layer Industrial World Model Stack with Cognitive Agent Layer. Cognitive agents observe plant state, generate candidate strategies, simulate them through all three prediction layers, and present ranked recommendations with explanations to operators. API strips between layers show the standardised service boundaries; what sits behind each API is a deployment decision.

3.2 Layer 1: Physical Dynamics ($\mathcal{L}_1$)

Core question: Given the current physical state $\mathbf{s}_t$ and control actions $\mathbf{a}_t$, what is the predicted physical state $\mathbf{s}_{t+\Delta}$?

$$\mathcal{L}_1 : (\mathbf{s}_t,\, \mathbf{a}_t,\, \Delta) \;\mapsto\; (\hat{\mathbf{s}}_{t+\Delta},\, \boldsymbol{\phi}_t) \tag{2}$$

where $\hat{\mathbf{s}}_{t+\Delta}$ is the predicted physical state vector and $\boldsymbol{\phi}_t$ is a vector of physical risk flags (e.g., overload, roping, cavitation, high vibration).

Reading Equation (2): What Layer 1 Does

Layer 1 answers: "Given what the plant looks like now ($\mathbf{s}_t$) and the control moves I am about to make ($\mathbf{a}_t$), what will the plant look like physically in Δ minutes?" It returns a predicted future state ($\hat{\mathbf{s}}_{t+\Delta}$) plus physical risk flags ($\boldsymbol{\phi}_t$) that indicate dangerous conditions such as mill overload, cyclone roping, or pump cavitation. This is the layer AI labs are building.

This is the layer that JEPA, V-JEPA 2 (9), Dreamer (6), MuZero (11), and Marble (14) are building. For industrial applications, Layer 1 implementations draw on three sources:

- **First-principles models:** Breakage matrices, population balance equations, Bond/Kick-/Rittinger energy models, fluid dynamic models.
- **Learned surrogates:** NARX networks, deep neural networks, and Gaussian process models trained on historian data and high-fidelity simulations.
- **Latent visual models:** JEPA-style vision encoders that learn to represent plant states (mill charge levels, froth texture, separator interface positions) from camera feeds without pixel-level prediction.

AI labs are investing substantially in this layer, and the industrial applications are significant. It is, however, necessary but not sufficient.

3.3 Layer 2: Operational KPI Dynamics ($\mathcal{L}_2$)

Core question: Given a physical state trajectory $\{\hat{\mathbf{s}}\}_t^{t+H}$ and planned control actions, what are the predicted operational KPIs over horizon H?

$$\mathcal{L}_2 : (\{\hat{\mathbf{s}}\}_t^{t+H},\, \mathbf{a}_{t:t+H},\, H) \;\mapsto\; (\hat{\mathbf{k}}_{t:t+H},\, v,\, \sigma_k) \tag{3}$$

where $\hat{\mathbf{k}}_{t:t+H}$ is the KPI time-series forecast (throughput, grade, recovery, energy, maintenance risk), v is a scalar economic objective, and σ_k is a confidence estimate.

Reading Equation (3): What Layer 2 Does

Layer 2 takes the physical state trajectory from Layer 1 and translates it into operational value: "Given how the plant will behave physically over the next H hours, what happens to throughput, concentrate grade, copper recovery, energy cost, and maintenance risk?" It returns KPI forecasts ($\hat{\mathbf{k}}$), a single economic score (v) that captures the net value of the plan, and a confidence estimate (σ_k) indicating how certain the prediction is. This is the layer that turns physics into business value.

Layer 2 translates physical state evolution into the metrics that operations actually optimises for. Existing digital twins (Emerson Mimic, ABB/Boliden Dymola) and APC systems implement significant portions of Layer 2 (3). The architectural contribution is wrapping these behind standardised APIs that agents can call programmatically for multi-scenario evaluation. These systems were designed for human dashboards and control room displays, not as composable prediction services for autonomous agents.

3.4 Layer 3: Socio-Technical Dynamics ($\mathcal{L}_3$)

Core question: Given a KPI trajectory and proposed action schedule, what are the predicted safety, compliance, human workload, and organisational side-effects?

$$\mathcal{L}_3 : (\hat{\mathbf{k}}_{t:t+H},\, \mathbf{a}_{t:t+H},\, \mathcal{C}) \;\mapsto\; (\rho,\, \mathbf{F},\, \omega,\, E,\, \Delta\mathcal{C}) \tag{4}$$

where $\mathcal{C}$ is the constraint knowledge base, $\rho \in [0, 1]$ is an overall risk score, $\mathbf{F}$ is a vector of binary safety and compliance flags, ω is an operator workload estimate, E is a natural language explanation of the risk assessment, and $\Delta\mathcal{C}$ is a set of proposed updates to the constraint knowledge base derived from observed operator behaviour.

Reading Equation (4): What Layer 3 Actually Does

Layer 3 takes three inputs and produces five outputs. In operational terms:

Inputs — what the system already knows at this point:

- $\hat{\mathbf{k}}_{t:t+H}$: The predicted KPI trajectory from Layer 2 — throughput, grade, recovery, energy, etc. over the planning horizon (e.g., the next 2 hours).
- $\mathbf{a}_{t:t+H}$: The proposed action schedule — the sequence of control moves the agent plans to make (mill speed change at minute 5, cyclone pressure adjustment at minute 12, etc.).
- $\mathcal{C}$: The constraint knowledge base — everything the system knows about what is acceptable: hard safety limits, contractual obligations, maintenance thresholds, alarm rate tolerances, shift patterns, environmental permits.

Outputs — what Layer 3 returns:

- ρ : A single risk score between 0 (no concern) and 1 (unacceptable). The headline number for comparing plans.
- $\mathbf{F}$: A vector of binary flags — one per constraint. Did we trip a safety limit? Violate an environmental threshold? Even if ρ is moderate, a single flag on a hard safety constraint is a veto.
- ω : Operator workload estimate — will this plan generate alarm floods or require attention beyond what is reasonable during the current shift?
- E : A natural language explanation: not just "risk is 0.73" but *why* — which constraints are active, what triggered them, and what the operator should understand.
- $\Delta\mathcal{C}$: Proposed updates to the constraint knowledge base itself, learned from operator overrides. For example: "Operators have overridden throughput increases above 3% during the first hour after shift change on 7 of the last 9 occasions. Proposed new constraint: limit autonomous throughput changes to $\leq$2% within 60 minutes of shift handover."

The critical architectural point: E requires the system to reason in language about risk (not just compute a number), and $\Delta\mathcal{C}$ requires the system to learn new constraints from operational experience (not just enforce a static rule set).

The inclusion of E is architecturally significant, not cosmetic. Regulatory frameworks including the EU AI Act classify autonomous control recommendations in high-risk industrial environments as systems requiring human oversight with explainable justification. A Layer 3 that produces $\rho = 0.73$ with no explanation of which constraints are active, why they were triggered, and what the operator should understand cannot satisfy these requirements in regulated jurisdictions. This technically requires a generative language capability at Layer 3.

The inclusion of $\Delta\mathcal{C}$ establishes that Layer 3 is not a static rule set but a learning constraint model. When an operator consistently overrides a recommendation in a specific operating context, that override encodes a constraint the formal model did not capture. $\Delta\mathcal{C}$ represents the system's capacity to detect, formalise, and incorporate these implicit constraints over time. This requirement eliminates static rule-based approaches from the viable Layer 3 solution space.

Layer 3 is rarely discussed in the world model literature, understandably, since it is domain-specific rather than foundational. It models how humans, procedures, regulations, and business systems respond to plant changes:

- **Alarm and workload dynamics.** A control strategy that reduces KPI variance might triple the alarm rate during ore hardness transitions. An operationally optimal plan that overwhelms the night-shift operator is not optimal.
- **Maintenance acceleration.** Running at aggressive throughput for 12 hours may trigger accelerated liner or pump failure that eliminates a week of production gains.
- **Business and contractual constraints.** Shipment specifications, SLAs, insurance triggers, environmental reporting thresholds, regulatory limits.
- **Organisational friction.** Persistent operator overrides are data: they indicate unmodelled constraints that no physics model captures.

Data sources include historian alarm logs, CMMS work orders and inspection records, production plans and contracts, safety reporting systems, and environmental monitoring data. BHP's published work on digital twins and AI in mining decision-making explicitly calls out these multi-dimensional constraints as essential to operational deployment (5).

3.5 The Knowledge Acquisition Problem

The formal definitions of $\mathcal{L}_2$ and $\mathcal{L}_3$ specify what each layer outputs but leave open a question that may be the hardest industrial AI problem of all: how does the knowledge that populates Layers 2 and 3 enter the system, and how does it remain current?

Layer 2 knowledge — the digital twin models, surrogate KPI predictors, and objective functions — typically requires months of engineering commissioning per plant. Model parameters must be fitted to specific ore bodies, equipment configurations, and operating envelopes. This is well-understood engineering work, but it is expensive and plant-specific.

Layer 3 knowledge is structurally harder. The constraints governing safe and acceptable operation are almost entirely encoded in unstructured text: P&IDs, HAZOP reports, contracts, shift handover notes, maintenance procedures, environmental permits, and — critically — operator institutional memory that has never been written down. Three technical requirements follow:

1. **Bootstrap ingestion.** $\mathcal{L}_3$ must be initialisable from unstructured documents without requiring a knowledge engineer to manually encode every constraint. Systems that require manual formalisation of each rule cannot deploy at scale across heterogeneous plants with thousands of pages of operating procedures.
2. **Override learning.** When an operator consistently overrides a recommendation in a specific situation, the override encodes an implicit constraint that the formal model did not capture. The system must record the override, identify the pattern across multiple instances, and update $\mathcal{C}$ accordingly. This is not a supervised learning problem — there is no labelled dataset of "constraint violated / not violated." It requires reasoning about *why* the override occurred from the operational context surrounding it.
3. **Cross-plant transfer.** A constraint learned at one operation (e.g., that aggressive throughput pushing before a scheduled reline accelerates liner failure risk) should transfer with appropriate modification to a structurally similar operation (e.g., that aggressive pushing before a SAG shell inspection carries analogous risk). This requires reasoning at the level of operational abstractions, not pattern matching on specific tag values.

A technology that cannot satisfy all three requirements will produce a Layer 3 that is accurate at deployment but degrades as operating conditions, personnel, equipment, and business constraints evolve — which is precisely the regime where the consequences of constraint violation are highest.

3.6 The Operator Co-Evolution Requirement

Layer 3 is not static. The socio-technical constraints governing a plant change continuously as operators gain experience with the autonomous system, as equipment ages, as ore bodies or reservoir conditions shift, and as business conditions evolve. A constraint knowledge base $\mathcal{C}$ that is comprehensive at deployment will drift from operational reality within months.

This creates a requirement beyond prediction: *continuous constraint learning from operational feedback*. Specifically:

- When an operator overrides an agent recommendation, the override and its surrounding context (plant state, time of shift, recent alarm history, verbal justification if recorded) must be captured and analysed. Over multiple instances, a pattern may emerge that identifies a constraint $c_{\text{new}} \notin \mathcal{C}$ that should be proposed for inclusion.
- When a predicted alarm rate proves inaccurate under specific operating conditions, the workload model within $\mathcal{L}_3$ must be recalibrated against the realised alarm data, not just globally but conditionally on the operating regime.
- When a maintenance risk prediction is contradicted by actual equipment behaviour (e.g., predicted liner life was significantly over- or under-estimated), the maintenance acceleration model must update its priors for that equipment class.

This co-evolution mechanism is architecturally distinct from both the prediction services ($\mathcal{L}_1$, $\mathcal{L}_2$) and the static constraint knowledge base. It is a feedback loop between the agent's $\mathcal{L}_3$ model and the operational record that runs on a slower timescale (days to weeks) than the planning cycle (minutes to hours). The $\Delta\mathcal{C}$ output in Equation (4) is the interface through which this co-evolution surfaces proposed constraint updates for review.

Systems that treat Layer 3 as a fixed configuration — however comprehensive at initial deployment — cannot satisfy this requirement. The co-evolution loop is what distinguishes an adaptive industrial world model from a static constraint checker.

3.7 The Cognitive Agent Requirement

The three-layer stack as defined above provides prediction services: given a state and proposed actions, each layer returns predictions about physical outcomes, KPI trajectories, or socio-technical risk. But prediction services alone do not produce autonomous behaviour. Something must *observe* the current plant state, *reflect* on what it means in context, *plan* candidate action sequences, *simulate* those candidates through all three layers, *evaluate* the results against multiple competing objectives, *explain* the reasoning to operators, and *learn* from the outcomes. This is the cognitive agent layer.

The cognitive agent requirement is not a minor integration detail. It is an architectural requirement that determines the class of technology capable of operating the 3-layer stack. Three properties make this clear:

1. **Multi-objective reasoning under uncertainty.** The three layers return predictions with confidence estimates, competing objectives (throughput vs. energy vs. maintenance risk vs.

contractual compliance), and constraint flags. The agent must reason over these trade-offs to produce a recommendation — not simply optimise a weighted sum. A plan that is physically suboptimal ($\mathcal{L}_1$) may be operationally superior ($\mathcal{L}_2$) and organisationally acceptable ($\mathcal{L}_3$), while the physically optimal plan triggers a contractual inspection requirement that costs two production days. Discovering this requires reasoning, not optimisation.

2. **Hypothesis generation from context.** The agent must generate diverse, plausible candidate strategies — not just perturb setpoints around the current operating point. When ore hardness changes, the space of useful responses includes feed rate reduction, mill speed adjustment, classification changes, flotation reagent modification, or combinations of these. Generating this candidate set requires understanding the process context and reasoning about what kinds of interventions address the specific situation, which is a cognitive capability, not a search algorithm.

3. **Grounded explanation and dialogue.** The E output from $\mathcal{L}_3$ (Equation 4) requires natural language explanation of risk assessments. But the cognitive agent's explanation requirement goes further: it must explain its *reasoning process* — why it generated these candidates, why it ranked them this way, what trade-offs it considered, and what it would recommend if the operator modifies a constraint. This is an interactive reasoning capability that enables the operator co-evolution described in Section 3.6.

These three properties — multi-objective reasoning, contextual hypothesis generation, and grounded explanation — collectively define a cognitive agent architecture. The agent must be grounded in engineering and physics (its reasoning must be calibrated against $\mathcal{L}_1$ and $\mathcal{L}_2$ predictions, not hallucinated) while also capable of the language-based constraint reasoning and explanation that $\mathcal{L}_3$ demands.

This rules out several classes of system that might initially appear sufficient:

- **Conventional optimisers** (MPC, LP/QP solvers) can solve the weighted-sum version of the multi-objective problem but cannot generate diverse hypotheses from context, cannot reason about novel constraints in natural language, and cannot explain their trade-off reasoning to operators.

- **Reinforcement learning agents** learn effective policies through trial but do not produce explanations, cannot reason about unstructured constraints ($\mathcal{L}_3$ content), and require extensive online or simulated experience for each new operating context.

- **Large language models alone** can reason about constraints and generate explanations but are not grounded in calibrated physical models. Without $\mathcal{L}_1$ and $\mathcal{L}_2$ grounding, their KPI predictions and physical reasoning will be unreliable.

- **Rule-based expert systems** can encode known constraints but cannot reason about novel situations, cannot learn from operator overrides, and cannot generate contextually diverse candidate strategies.

The architecture that satisfies all three properties is a *multi-agent cognitive system*: a collection of specialised agents (process optimisation, maintenance risk, safety compliance, production scheduling) coordinated through a shared world model stack, where each agent follows an observe–reflect–plan–act cycle grounded in the three prediction layers. Individual agents contribute domain-specific reasoning; the orchestration layer resolves conflicts, manages authority boundaries, and escalates to human oversight when agent consensus cannot be reached.

This cognitive agent architecture is what sits above the three-layer stack in Figure 1 — the component currently labelled "Recommendation." It is not an optional wrapper. It is the reasoning engine that makes the prediction services operationally useful, and its requirements (grounded in physics, capable of language-based reasoning, adaptive through co-evolution) define the technology needed to complete the industrial world model architecture.

4 Case Study: Copper Grinding-Flotation Circuit

4.1 Circuit Description

A copper grinding-flotation circuit provides an effective validation domain for the three-layer stack. The simplified flow: crushed ore enters SAG and ball mills, hydrocyclones classify the mill discharge, and rougher, cleaner, and scavenger flotation cells separate copper-bearing minerals from gangue. Key control variables include mill speed, feed rate, water addition, cyclone pressure, air flow, reagent dosing, and froth depth (4).

The circuit is rich in Layer 1, Layer 2, and Layer 3 content: complex physical dynamics at the mill and cyclone, significant economic leverage at the flotation stage, and substantial socio-technical risk from alarm floods, liner wear, and contractual grade specifications.

4.2 Layer 1 in the Grinding-Flotation Context

The physical dynamics layer captures material and equipment behaviour: harder ore at constant mill speed produces coarser particle size distributions; cyclone pressure and water flow change cut size and the underflow/overflow split; froth depth, air rate, and bubble size affect attachment probability and recovery by mineral species.

Implementation draws on breakage matrices and population balance models for grinding (2), classification models for cyclones, and learned surrogates for flotation grade and recovery (10; 8). In a JEPA-era implementation, computer vision on froth surfaces, mill charge states, and sump levels adds latent features encoding critical plant states (overload, poor froth stability, high circulating load) without requiring pixel-level prediction. This is the pattern V-JEPA 2 is designed for.

4.3 Layer 2 in the Grinding-Flotation Context

The KPI dynamics layer translates physical state evolution into the metrics that matter operationally. Using process simulation tools and AI surrogate models (10), Layer 2 answers questions such as:

> *If I raise mill speed by 2%, relax cyclone pressure by 15 kPa, and deepen rougher tails level by 5 cm, what happens to plant throughput, concentrate grade 20 minutes from now, energy per tonne, and risk of process instability?*

The ABB/Boliden Aitik case study demonstrates this pattern at production scale: a digital twin coupled with APC enables testing of new control strategies in simulation before live deployment, with measured improvements in throughput and energy efficiency (3).

4.4 Layer 3 in the Grinding-Flotation Context

The socio-technical layer evaluates operational acceptability. Three examples illustrate what Layer 3 adds beyond Layer 2:

1. A control policy that reduces KPI variance during an ore hardness event may generate three times more alarms per hour. Layer 3 flags this and estimates the probability of operator override or manual intervention.
2. Running at aggressive throughput for 12 hours may trigger accelerated mill liner failure. Layer 3 accesses CMMS records to estimate maintenance acceleration and integrates it into the risk score.
3. A particular grade profile, optimal in economic terms, may jeopardise a shipment specification with contractual consequences. Layer 3 checks production plans and contract databases.

5 Case Study 2: Upstream Oil & Gas Separation Train

5.1 Train Description

The same three-layer structure applies directly to an upstream oil and gas separation train, with wells, headers, and three-phase separators replacing mills and flotation cells. The simplified flow: wells produce through a gathering network into an inlet manifold, through optional heating, into three-phase separator vessels that produce gas (to compression and export), oil (to stabilisation and storage), and water (to treatment or re-injection). Key control variables include well choke positions, separator pressure, gas/oil/water level setpoints, interface level, temperatures, chemical injection rates, and compressor load.

The separation train is a good second validation domain because its Layer 3 content is distinctly different from mining: pressure vessel integrity, flaring limits, and environmental emissions constraints are hard safety boundaries that have no mining equivalent, and the consequences of Layer 3 violations are immediate and regulatory rather than contractual.

5.2 Layer 1: Fluid and Equipment Dynamics

The physical dynamics layer captures how fluids behave in the separation train. Opening well chokes increases inlet pressure and liquid and gas rates; slugs from wells propagate into the separator. In the separator vessel, gas disengages and rises, oil forms a middle layer, and water accumulates below; interface and total liquid levels are controlled by dump valves. Poor level and interface control leads to carry-over (liquid in gas) or carry-under (oil in water), which upsets downstream compressors or water treatment. Pressure, temperature, and residence time govern separation efficiency, gas quality, and foaming and emulsion behaviour.

Current implementations use first-principles or Petro-SIM/HYSYS-style process models of the separator and upstream network, dynamic twins for operator training and control logic validation, and emerging reinforcement learning controllers for three-phase separator level and flow control (12; 13).

5.3 Layer 2: Production, Quality, and Economics

Layer 2 translates physical separation dynamics into production, quality, and cost metrics: oil production rate and stabilised oil quality (RVP, BS&W at train outlet); gas export rate and quality (heating value, contaminants, dew point); water quality against discharge or re-injection specifications; and energy use (compressor load, heater duty), flare volume, and associated emissions costs.

Yokogawa, KBC, and others already run continuous simulation and optimisation on digital twin engines to maximise production subject to constraints. Layer 2 here answers: *If I increase well choke positions and adjust separator pressure setpoints, how much more oil and gas do we produce, and what happens to flare volume and energy cost over the next four hours?*

5.4 Layer 3: Safety, Integrity, and Environmental Constraints

Layer 3 for a separation train carries constraints that are structurally different from those in mineral processing. Safety and integrity constraints include separator and vessel pressure limits, PSV setpoints, maximum liquid levels, minimum residence time, and compressor and flare system limits. Erosion and corrosion constraints in pipes and vessels apply continuously. Environmental and contractual constraints include flare volume or duration limits, emissions targets, and export gas and oil quality specifications with contractual penalties.

Operational behaviour patterns also differ: alarm and trip responses on separation trains are shaped by start-up and shutdown procedures, HAZOP-derived operating envelopes, and integrity and corrosion models. Historical alarm, trip, and override data encode where operations become sensitive, exactly the signal Layer 3 is designed to learn from.

5.5 What Differs Across Domains

The two case studies illustrate the domain-specificity of Layers 2 and 3 while Layer 1 remains structurally identical. Table 3 summarises the key differences.

Table 3: Layer content by domain: copper grinding-flotation versus upstream oil & gas

Layer	**Copper Grinding-Flotation**	**Oil & Gas Separation Train**
1 (Physical)	Breakage matrices, population balance, cyclone classification, flotation froth dynamics	Fluid phase behaviour, separator vessel dynamics, slug propagation, compressor response
2 (KPI)	Throughput (t/h), concentrate grade (%Cu), recovery (%), energy (kWh/t)	Oil rate (bbl/day), gas export (MMscfd), BS&W, flare volume (MMscf), energy cost ($/day)
3 (Socio-Technical)	Alarm floods, liner and pump wear, shipment grade specifications, shift-change workload	Vessel pressure limits, PSV setpoints, flare duration limits, emissions thresholds, corrosion envelopes

The Layer 1 API surface (`/world/state`, `/world/simulate-step`, `/world/rollout`) is identical across both domains. The domain-specific content is entirely within Layers 2 and 3, which is precisely where industrial expertise sits.

6 End-to-End Example: Autonomous Agent Plans +5% Throughput

The following ten-step sequence describes one planning cycle for an agent tasked with increasing throughput by 5% on a copper grinding-flotation circuit while maintaining grade, recovery, and safety constraints.

Step 1. Sense current state. The agent reads live plant state: mill speed, power, feed tonnage, cyclone pressure, sump levels, froth levels, APC mode flags, and current KPIs.

Step 2. Define goal and constraints. Target: +5% throughput over a two-hour horizon. Hard constraints: grind size within specification, recovery above threshold, no safety or process envelope violations.

Step 3. Generate candidate strategies. The agent creates three plausible strategies: (A) +3% mill feed with +2% mill speed and cyclone pressure adjustment; (B) +5% feed at constant speed with classification and water changes; (C) moderate feed increase with flotation changes to protect recovery.

Step 4. Simulate through Layer 1. For each strategy, call $\mathcal{L}_1$ to predict state trajectories over 30–60 minutes. Strategies that hit physical limits (overload, roping, cavitation) are rejected.

Step 5. Evaluate KPIs via Layer 2. Surviving strategies pass to $\mathcal{L}_2$. The layer returns forecast time series for throughput, grind, recovery, grade, and energy per tonne. Strategies not meeting KPI constraints are eliminated.

Step 6. Check socio-technical constraints via Layer 3. Top candidates pass to $\mathcal{L}_3$ for evaluation against safety flags, predicted alarm rate and operator workload, expected maintenance acceleration, and overall risk score.

Step 7. Compare with APC baseline. The agent queries the existing APC system for its recommended moves under the same +5% target and runs the identical evaluation, producing a three-way comparison.

Step 8. Present to operator. The agent presents the recommended plan with expected improvements, predicted risk status, and a clear action sequence. The operator can approve, modify, or reject.

Step 9. Execute with guardrails. On approval, setpoint trajectories are sent to APC/DCS via OPC-UA, not as direct valve commands. APC enforces its own move limits and interlocks. If the operator engages manual mode, the agent immediately suspends setpoint issuance.

Step 10. Learn from outcome. The agent logs plan details, Layer 1–3 predictions, operator decision, and actual plant response. Predicted versus actual KPIs are compared. Surrogate models, risk scores, and operator preference priors are updated for the next cycle.

Over many cycles, the world model stack calibrates to the specific plant and ore body. Autonomy envelopes expand as the agent demonstrates reliable prediction accuracy on low-risk micro-adjustments.

7 API Architecture

The three-layer stack exposes clean API surfaces that agents call programmatically. Each layer is a service; orchestration logic sits above all three. Table 4 lists the principal endpoints.

Table 4: 3-Layer world model API surface

Endpoint	Layer	Input	Output
`GET /world/state`	1	None (live plant)	State vector: tags, latent features, health flags
`POST /world/simulate-step`	1	State + control moves + horizon	New state + physical risk flags
`POST /world/rollout`	1	State + action sequence + H	Full state trajectory over H
`POST /kpi/forecast`	2	State trajectory + actions	KPI time series + confidence
`POST /kpi/objective`	2	KPI trajectories	Scalar objective + decomposed contributions
`POST /kpi/suggest-actions`	2	Current state + goals	APC/RL recommended control sequence
`POST /constraints/evaluate`	3	State + KPI trajectories + action schedule	Safety flags, alarm rate, maintenance risk, risk score
`POST /workload/estimate`	3	Proposed plan	Operator cognitive load estimate
`GET /constraints/guardrails`	3	None	Safety rules and operating envelopes

The critical design principle: agents never interact with raw historian tags or vendor-specific implementations. They interact with standardised state abstractions, prediction services, and constraint evaluators. What sits behind each endpoint (Dymola, Emerson Mimic, a NARX surrogate, a JEPA-style visual model, or an AMI foundation model) is a deployment decision, not an architectural one.

8 Alignment with LeCun's AMI Architecture

LeCun's publicly described blueprint for autonomous machine intelligence specifies four main components: a perception module, a world model, a cost/value module, and an actor/policy (7). The three-layer industrial stack maps cleanly onto this architecture. Table 5 shows the mapping.

Table 5: Mapping LeCun's AMI components to the 3-layer industrial stack

LeCun/AMI Component	**3-Layer Mapping**	**Industrial Implementation**
Perception Module	Shared `PlantState` abstraction	Perception agent fuses DCS tags, vibration, lab data, and cameras into latent state $\mathbf{z}_t$ with decoded KPI view
World Model	$\mathcal{L}_1$ (physical) + $\mathcal{L}_2$ (KPI dynamics)	`IWorldModelProvider` interface: `encode_observation`, `predict_dynamics`, `predict_reward`. Vendor-agnostic registry
Cost/Value Module	$\mathcal{L}_2$ objective scoring + $\mathcal{L}_3$ constraint evaluation	`TrajectoryEvaluator`: decomposes economic value, safety score, maintenance risk, and compliance risk
Actor/Policy	Agent planning strategies	`WorldModelPlanningStrategy`: generates N candidates, simulates through all three layers, returns ranked plans

The strategic implication of this mapping is direct. AMI Labs' JEPA-style models are Layer 1 providers. They plug into the industrial stack via a standardised interface. Layers 2 and 3 are where domain expertise in industrial operations creates defensible and lasting value, and where AMI is unlikely to invest deeply.

Key architectural principle: The world model provider interface is deliberately vendor-agnostic. AMI models, open-source Dreamer variants, and in-house trained surrogates all implement the same contract: `encode_observation(`o_t`)` $\rightarrow z_t$, `predict_dynamics(`z_t`, `a_t`)` $\rightarrow z_{t+1}$, `predict_reward(`z_t`, `a_t`)` $\rightarrow r_t$. Swapping providers is a configuration decision, not a re-architecture.

9 Progressive Autonomy

Deployment of world-model-based agents in industrial settings follows four phases, each building on the calibration data generated by the previous one. Table 6 defines the phases.

Table 6: Progressive autonomy phases for world-model-based industrial agents

Phase	**World Model Role**	**Agent Authority**	**Human Role**
Shadow Mode	Predictions run in parallel with existing control. Logs cases where the world model would have recommended different actions.	Observe and predict only. No control authority.	Full control. Reviews prediction accuracy reports.
Advisory Mode	Recommendations generated with simulated outcomes through all three layers.	Recommend setpoint changes, feed adjustments, maintenance work orders.	Approves, modifies, or rejects all recommendations.
Bounded Autonomous	Full three-layer simulation within defined autonomy envelopes.	Executes low-risk actions autonomously. Escalates out-of-envelope situations.	Sets autonomy boundaries. Handles escalations.
Collaborative Autonomous	Continuous calibration and envelope expansion based on outcomes.	Manages routine operations. Proposes envelope expansions.	Strategic oversight. Focuses on exceptions and boundary management.

The phased structure is architecturally necessary. Each phase generates the calibration data (predicted versus actual outcomes, operator override patterns, constraint activation frequencies) that the next phase requires to expand autonomy responsibly. Shadow mode cannot be skipped: it provides the ground-truth comparison data that allows the operator community to verify model accuracy before advisory mode begins.

9.1 Technical Gate Criteria

Each phase transition requires the system to demonstrate specific capabilities that cannot be assessed through manual review alone. These gate criteria define what the system must *prove* before progressing:

- **Shadow → Advisory.** Requires demonstrated $\mathcal{L}_2$ prediction accuracy within defined tolerance of actual outcomes over a statistically significant number of operating cycles across distinct operating regimes (e.g., varying ore hardness, feed rates, equipment states). The system must produce this evidence autonomously through continuous predicted-versus-actual comparison, not through periodic manual audit.
- **Advisory → Bounded Autonomous.** Requires $\mathcal{L}_3$ constraint model coverage verified against the historical operator override database. No override in the evaluation period should

remain unexplained by the current constraint model $\mathcal{C}$. If overrides occur that the model cannot account for, they indicate missing constraints that must be resolved (via $\Delta\mathcal{C}$) before autonomous authority is granted.

- **Bounded → Collaborative Autonomous.** Requires demonstrated $\Delta\mathcal{C}$ learning: the system must show evidence that operator overrides have resulted in constraint model updates, and that subsequent encounters with similar operating situations were handled correctly without override. This gate verifies that the co-evolution mechanism (Section 3.6) is functioning and that the constraint model is converging rather than drifting.

Each gate criterion requires capabilities — autonomous prediction accuracy tracking, constraint coverage verification, and learning verification — that are not present in existing digital twin platforms or APC systems. Stating them formally makes the architectural requirements for each autonomy phase precise and testable.

10 Strategic Implications

10.1 For AI Research Labs

Layer 1 models require two additional architectural layers to be operationally deployable. The architecture defines a natural division: research labs build the physical foundation, industrial platforms build the operational and socio-technical layers, and enterprises contribute domain context and production validation. Foundation world models plug into Layer 1 through the `IWorldModelProvider` interface. What sits above that interface (KPI dynamics, constraint evaluation, safety envelopes, multi-agent orchestration, human oversight) is outside the scope of a world model research programme by design.

10.2 For Industrial Enterprises

Layers 2 and 3 can be built today using existing digital twins, APC systems, alarm historians, and CMMS data. The building blocks are already in production at most major industrial operations. What has been missing is the architectural framework that exposes them as agent-callable prediction services. Building that framework now means Layers 2 and 3 are in place when foundation world models mature, and no re-architecture is required to plug them in.

The two case studies in this paper illustrate the same underlying point: the domain-specific content in Layers 2 and 3 accumulates through production deployments, domain engineering, and operational experience over years. That is not content a general-purpose AI programme produces.

10.3 For Industrial Platform Providers

Platform providers who own Layers 2 and 3 integration (multi-agent orchestration, safety envelopes, OT connectivity, and human oversight patterns) retain value regardless of which foundation model provider supplies Layer 1. AMI, DreamerV3, or an in-house surrogate all sit behind the same interface. The vendor-agnostic design means the industrial stack is not exposed to provider risk as the foundation model landscape develops.

11 Conclusion

AMI Labs, Meta, and World Labs are building the physical reasoning layer that industrial autonomy requires: learning physical cause and effect from raw sensory data without supervision.

It is one layer of three.

An AI that understands that increasing mill speed will coarsen particle size is not ready to operate a copper concentrator. An AI that also knows what that coarsening does to concentrate grade and recovery, what that grade change means for tomorrow's shipment specification, and whether the control sequence required will overwhelm the night-shift operator: that system is closer to deployable.

The three-layer industrial world model stack provides the complete architecture. Layer 1 (physical dynamics) is being built by AI research labs. Layers 2 (operational KPI dynamics) and 3 (socio-technical constraints) encode operational and organisational intelligence that industry has accumulated but not yet exposed as composable prediction services for autonomous agents.

This paper has argued that the full stack requires more than prediction at three levels. Layer 3 must produce explainable risk assessments to satisfy regulatory requirements for human oversight, and it must learn from operational feedback — particularly operator overrides — to remain accurate as plant conditions evolve. The knowledge acquisition problem (how Layer 2 and 3 knowledge enters the system from unstructured documents, operational experience, and cross-plant transfer) is perhaps the hardest unsolved challenge in the space. And the progressive autonomy framework requires formal technical gate criteria that the system must satisfy autonomously before advancing to greater authority.

These requirements collectively define a system that spans fundamentally different technology categories: physics-grounded simulation for Layer 1, calibrated KPI surrogates for Layer 2, language-grounded constraint reasoning with adaptive learning for Layer 3, and cognitive agents capable of observe–reflect–plan–act reasoning grounded in both engineering physics and natural language. No single existing technology satisfies all four requirements. The architecture requires their composition.

The cognitive agent layer is where the prediction stack becomes operational. Without agents that can generate hypotheses, reason over multi-objective trade-offs, explain their decisions, and learn from operator feedback, the three prediction layers remain passive services — useful, but not autonomous. With cognitive agents grounded in calibrated world models, the architecture enables a progressive path from shadow mode through advisory to bounded autonomy, each phase building on demonstrated prediction accuracy and constraint learning.

Neither side can build the full stack alone. AI labs bring the physics. Industry brings the operational context, the constraint knowledge, and the co-evolution data that only accumulates through production deployment. Cognitive agent architectures provide the reasoning layer that connects physical prediction to operational decision-making. The composable architecture defines how these components connect, with clean API boundaries that let each party focus on what they do best.

References

[1] AMI Labs. (2026). AMI Labs: Real World. Real Intelligence. *amilabs.xyz*. See also: Coldewey, D. (2026). Yann LeCun's AMI Labs raises $1.03B to build world models. *TechCrunch*, techcrunch.com/2026/03/09/yann-lecuns-ami-labs-raises-1-03-billion-to-build-world-models/.

[2] Austin, L. G., Klimpel, R. R., & Luckie, P. T. (1984). *Process Engineering of Size Reduction: Ball Milling*. Society of Mining Engineers.

[3] ABB Industrial Software. (2024). Case Study: Digital Twin with Advanced Process Control at Boliden Aitik Mine. Technical report, ABB.

[4] Bascur, O. A. (2025). The Rise of AI-based Digital Twin Mineral Processing Plant Operations. In *Proceedings of Flotation '25*.

[5] BHP. (2025). The Role of Digital Twins and AI in Enhancing Decision-Making in the Mining Industry. BHP Insights.

[6] Hafner, D., Lillicrap, J., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2023). Dream to Control: Learning Behaviors by Latent Imagination. In *International Conference on Learning Representations*.

[7] LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence. Meta AI Research.

[8] Li, Y. et al. (2025). Artificial Intelligence of Mineral Processing Processes. *Resources, Conservation and Recycling*.

[9] Meta AI. (2025). Introducing the V-JEPA 2 World Model and New Benchmarks for Physical and Embodied AI. *ai.meta.com/blog/v-jepa-2-world-model-benchmarks/*.

[10] Saxen, H. et al. (2024). Developing Deep Learning Surrogate Models for Digital Twins in Mineral Processing. *Minerals Engineering*.

[11] Schrittwieser, J. et al. (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588(7839), 604–609.

[12] Yokogawa. (2025). Facility-Wide Real-Time Optimisation for Oil & Gas Upstream with Process Digital Twin. *yokogawa.com*.

[13] Honeywell. (2024). Essential Digital Twins in Upstream Oil & Gas Production Operations. Technical report, Honeywell Process Solutions.

[14] World Labs. (2025). Marble: Explorable 3D Worlds and the World API. *worldlabs.ai/blog/announcing-the-world-api*.

Contact: Pieter van Schalkwyk, CEO, XMPro Inc. | pvs@xmpro.com | For commercial enquiries contact Gavin Green, VP Strategic Solutions: gavin.green@xmpro.com